



Infraestructura & Seguridad

Plataforma de mantenimiento industrial — cómo está montado el sistema por dentro

 Pendiente

 Asignado

 En ejecución

 Ejecutado

 Cerrado

Equipo: **Rosio Ametller · Jaime Osorio · Ramón Hernández**

Duración objetivo: **~10 minutos**

Agenda

01 · Infraestructura

Vista general — 3 capas desacopladas

Frontend — Next.js 16 en Vercel

Backend / API — NestJS 11 en Railway

Base de datos — Supabase PostgreSQL

Qué tecnología usamos y dónde corre cada pieza.

02 · Seguridad

Autenticación — JWT de Supabase

Autorización — roles y mínimo privilegio

Aislamiento multi-tenant

Evidencias — signed URLs

Secretos, red y RLS

Proceso — Gitflow y PRs

01

Infraestructura

Tres capas independientes: cada una se despliega, escala y falla por separado.

Vista general de la arquitectura

Frontend · SSR

Next.js 16 (App Router)
React 19
TanStack Query
shadcn/ui + Tailwind 4
Deploy: Vercel

Backend · API REST

NestJS 11 + TypeScript
Prisma 6 (ORM)
Auth JWT (Bearer)
Validación global de DTOs
Deploy: Railway

Datos · Servicios

Supabase
· PostgreSQL
· Auth (JWT)
· Storage (evidencias)
Región: East US

- ▶ El frontend nunca habla directo con la base: todo pasa por la API.
- ▶ Comunicación HTTPS + token JWT en cada request → un único punto de entrada controlado.

Frontend — Next.js 16 + Vercel

- ▶ Next.js 16 (App Router) + React 19, rendering en servidor.
- ▶ Protección de rutas en el servidor:
 - middleware valida la sesión (supabase.auth.getUser) en cada request
 - redirige a /login si no hay sesión, antes de renderizar
- ▶ Sesión en cookies vía @supabase/ssr (no localStorage → menos XSS).
- ▶ TanStack Query para estado de servidor y caché.
- ▶ shadcn + Tailwind 4 · formularios validados con zod.
- ▶ authFetch adjunta el token como Bearer (auto-refresh + retry en 401).
- ▶ El build de producción falla si faltan las env vars de Supabase.

Por qué importa

Sin lógica de negocio ni llaves secretas en el cliente.

Protección server-side: no depende de 'esconder un botón'.

Cookies > localStorage para la sesión.

Token refrescado de forma transparente.

producto/tract-front (paquete trackt-front)

Backend / API — NestJS 11

- ▶ NestJS 11 + TypeScript: arquitectura modular (un módulo por dominio).
- ▶ Prisma 6 como ORM → PostgreSQL (queries tipadas, anti-inyección).
- ▶ Validación global con ValidationPipe:
 - whitelist → descarta campos no declarados en el DTO
 - transform → convierte y valida tipos antes de la lógica
- ▶ Ningún dato entra a la API sin pasar por un filtro.
- ▶ Deploy en Railway (Node).

Módulos de dominio

auth · tenant
equipos · usuarios
ordenes · tickets
evidencias
inventario
notificaciones

9 módulos aislados → reglas de seguridad por módulo

Base de datos — Supabase PostgreSQL

- ▶ PostgreSQL administrada por Supabase (región East US).
- ▶ Conexión vía pooler (modo transacción, 6543) en runtime.
- ▶ Conexión directa (5432) sólo para migraciones.
- ▶ sslmode=require → tráfico a la DB siempre cifrado.
- ▶ tenant_id en TODAS las tablas de negocio → multi-tenant desde el diseño.

Modelo de datos

Tenant · Equipo
OrdenTrabajo · Ticket
Evidencia
Repuesto · InventarioStock
ReservaRepuesto
MovimientoInventario
Notificacion

Cubre todo el flujo del taller

02

Seguridad

Defensa en profundidad: varias capas, no una sola.

Autenticación — ¿quién eres?

- ▶ Auth delegada a Supabase Auth (no manejamos passwords).
- ▶ Cada request trae Authorization: Bearer <JWT>.
- ▶ AuthGuard en la API valida antes de cualquier endpoint:
 - extrae el token del header
 - lo valida contra Supabase (auth.getUser)
 - carga el perfil: rol + tenant desde 'profiles'
 - inyecta el usuario en el request
- ▶ Sin token válido o sin perfil → 401.
- ▶ Caché de perfiles en memoria (TTL 5 min).

Flujo del token

- 1 · Login en Supabase → JWT firmado
- 2 · Front guarda sesión en cookie
- 3 · authFetch envía Bearer a la API
- 4 · AuthGuard valida y resuelve perfil
- 5 · Endpoint se ejecuta con el usuario

No inventamos criptografía propia.

Autorización — ¿qué puedes hacer?

- ▶ 3 roles: admin · jefe_taller · mechanic. RolesGuard + @Roles(...) por endpoint.
- ▶ Modelo de mínimo privilegio — reglas en el código, no en la UI.

Acción sobre un ticket	Roles permitidos
Listar / ver	admin · jefe_taller · mechanic
Asignar / reasignar	admin · jefe_taller
Iniciar / finalizar	mechanic (el asignado)
Validar / cerrar	admin

Aislamiento multi-tenant

- ▶ Cada usuario pertenece a un tenant (taller).
- ▶ TenantService resuelve el tenant_id desde el TOKEN...
- ▶ ...nunca desde lo que mande el cliente.
- ▶ Toda consulta filtra por tenant_id.
- ▶ Un taller jamás ve datos de otro.

Ataque que cierra

Cambiar un ID en la URL para ver datos de otro taller.

Aunque lo intenten, la API siempre filtra por el tenant de la sesión autenticada.

El tenant no es un parámetro: se deriva del token.

Evidencias / archivos — signed URLs

- ▶ El archivo nunca pasa por nuestra API.
- ▶ 1 · Cliente pide URL → API valida rol, tamaño y tipo
 - genera URL firmada (TTL 60 s)
- ▶ 2 · Cliente sube directo a Supabase Storage
- ▶ 3 · Cliente confirma → API verifica que existe y registra
- ▶ Descarga con URL firmada temporal (TTL 5 min).

Controles

MIME whitelist: jpg · png · webp

Tamaño máximo: 5 MB

Ruta scopeada: tenant_id/ticket_id/uuid.ext

Descarga firmada y temporal (5 min)

Service-role key SÓLO en el servidor

Acceso por rol + asignación al ticket

Secretos, red y RLS

- ▶ .env en .gitignore; sólo se versiona .env.example sin valores.
- ▶ Service-role key y DB password sólo en el entorno del servidor.
- ▶ RLS habilitado en Supabase → segunda capa a nivel de base.
- ▶ HTTPS extremo a extremo · SSL obligatorio hacia la DB.

Defensa en profundidad

Para que un dato salga indebidamente tendría que fallar TODO:

- 1 · validación de input
- 2 · guard de auth (JWT)
- 3 · guard de roles
- 4 · filtro de tenant
- 5 · RLS en la base

Varias rejas, no una sola.

Proceso — Gitflow & PRs

- ▶ main protegida: nada se mergea sin Pull Request revisado.
- ▶ Una rama = un ticket de Linear = un PR · squash merge.
- ▶ Conventional Commits + Conventional Branches con ID de Linear.
- ▶ Trazabilidad automática: la rama mueve el estado del ticket.
- ▶ La seguridad también es proceso: reduce el riesgo de código sin revisar en producción.

En resumen

Infraestructura

3 capas desacopladas:

Next.js (Vercel) · NestJS (Railway) · Supabase

Cada una desplegada y escalada aparte.

El front nunca toca la base: todo por la API.

Seguridad en capas

validación → JWT → roles → multi-tenant → signed URLs → RLS

Sin secretos en el repo · HTTPS · mínimo privilegio

Defensa en profundidad de punta a punta.